OFFERO

Contexto del proyecto

Documento base para el equipo y para trabajar con asistentes de IA

Propósito
Definir la idea de Offero, las herramientas técnicas, la estructura inicial, los datos del sistema y la ruta de módulos para construir y validar el producto mínimo viable (MVP).

Versión de planificación · 21 de octubre de 2026

Contexto del proyecto — Offero

Instrucción de uso: Peguen este documento al inicio de cada conversación nueva con la IA. Actualícenlo cada vez que terminen o publiquen un módulo.

1. Proyecto

Nombre: Offero

Descripción: Offero es una plataforma web que conecta a personas y empresas que necesitan productos o servicios con negocios y proveedores que pueden ofrecerlos. Los compradores publican solicitudes y reciben cotizaciones que pueden comparar según precio, tiempo de entrega y condiciones, para elegir la propuesta que mejor se adapte a sus necesidades.

No incluye:

- Pagos integrados dentro de la plataforma en la primera versión.
- Aplicación móvil nativa para Android o iOS.
- Microservicios o una arquitectura de backend independiente.
- Funcionalidades avanzadas de inteligencia artificial.
- Expansión a múltiples países antes de validar el mercado inicial.
- Funcionalidades que no sean necesarias para validar el producto mínimo viable (MVP).

Repositorio: Pendiente de crear o agregar el enlace del repositorio del equipo en GitHub.

Aplicación publicada: Pendiente de publicar y agregar el enlace de Vercel.

2. Nivel del equipo

Somos estudiantes principiantes. Sabemos fundamentos de programación en Python y Java, programación orientada a objetos (clases, encapsulamiento, herencia), listas y HTML básico. No sabemos bases de datos ni desarrollo web con backend. Nunca hemos publicado una aplicación. Trabajamos en Windows con Visual Studio Code.

La IA debe explicar los pasos de forma progresiva, justificar las decisiones técnicas y evitar introducir herramientas o conceptos avanzados que no sean necesarios. Antes de modificar archivos existentes, debe revisar la estructura y el código actual del proyecto. Las instrucciones de terminal deben ser compatibles con Windows.

3. Integrantes

Integrante	Usuario de GitHub	Responsable de
Iker Esteban Fuentes Torres	https://github.com/fuentesiker	Desarrollador
Diego Alejandro Cely Ruiz	https://github.com/diegodani2030	Desarrollador
Daniel Alejandro Guevara Rosas	https://github.com/Arven1203-art/POO	Desarrollador

4. Herramientas elegidas

- **Frontend:** Next.js con React y Tailwind CSS.
- **Backend:** Lógica de servidor y API de Next.js, utilizando JavaScript. Supabase proporcionará servicios de autenticación, almacenamiento y comunicación en tiempo real cuando sean necesarios.
- **Base de datos:** PostgreSQL mediante Supabase.
- **Servicios de base de datos:** Supabase Auth para autenticación, Supabase Storage para archivos y Supabase Realtime cuando se necesite actualizar información en tiempo real.
- **Publicación:** Vercel para la aplicación web y Supabase para la base de datos y los servicios asociados.
- **Control de versiones:** Git y GitHub.
- **Editor:** Visual Studio Code.
- **Lenguaje principal:** JavaScript. TypeScript podrá evaluarse en el futuro, pero no forma parte de la configuración inicial elegida.

No debemos incorporar otro framework de frontend, otra base de datos o un backend independiente sin justificar primero por qué es necesario y acordarlo como una decisión del proyecto.

5. Forma de organizar el código

Utilizaremos funciones y componentes de React, organizados por responsabilidad, en lugar de construir toda la aplicación mediante clases.

- Componentes de React: para construir interfaces reutilizables, como encabezados, formularios, tarjetas de negocios, solicitudes y cotizaciones.
- Funciones de JavaScript: para procesar datos, validar información y ejecutar operaciones.
- Funciones de servidor y API de Next.js: para realizar las operaciones que necesiten ejecutarse de forma segura en el servidor.
- Módulos de servicios: para centralizar las operaciones relacionadas con Supabase y evitar repetir código.
- Clases: no serán el mecanismo principal de organización. Solo se utilizarán si existe una necesidad concreta que lo justifique.

La lógica de permisos, las validaciones importantes y las operaciones sensibles deben verificarse en el servidor o mediante las políticas de seguridad de Supabase, no únicamente en el frontend.

6. Datos que guarda el sistema

La base de datos será PostgreSQL mediante Supabase. Las siguientes son las tablas propuestas para el MVP. Sus campos definitivos y restricciones deberán ajustarse al diseñar e implementar cada módulo.

Nombre	Campos principales	Se relaciona con
profiles	id, full_name, email, role, created_at	auth.users de Supabase; negocios, solicitudes, cotizaciones y reseñas
businesses	id, owner_id, name, description, category, location, contact_info, created_at	profiles, categories
categories	id, name, description	businesses; solicitudes si se utiliza una categoría para clasificarlas
requests	id, client_id, category_id, title, description, quantity, location, deadline, status, created_at	profiles, categories, quotes
quotes	id, request_id, business_id, price, delivery_time, conditions, valid_until, status, created_at	requests, businesses
messages	id, sender_id, receiver_id, request_id, content, created_at, read_at	profiles, requests
reviews	id, reviewer_id, business_id, request_id, rating, comment, created_at	profiles, businesses, requests
reports	id, reporter_id, target_type, target_id, reason, status, created_at	profiles; el elemento reportado se identifica mediante su tipo y referencia

Consideraciones para la base de datos

- `profiles` complementará la autenticación de Supabase y almacenará el nombre y el rol de cada usuario.
- Los roles iniciales serán `client`, `provider` y `admin`.
- Un proveedor podrá tener un perfil comercial asociado a su cuenta.
- Una solicitud podrá recibir varias cotizaciones de distintos proveedores.
- Cada cotización estará vinculada a una solicitud y a un negocio.
- Las reseñas deberán corresponder a una experiencia real, según las reglas que definamos para evitar valoraciones falsas.
- Las tablas deben tener claves primarias, claves foráneas, restricciones y políticas de acceso apropiadas.
- Los campos exactos, los estados permitidos y las reglas de integridad se definirán durante el diseño de la base de datos.
- No se deben almacenar contraseñas directamente en `profiles`; la autenticación será gestionada por Supabase Auth.

7. Estructura de carpetas

La siguiente estructura es la propuesta inicial para Next.js utilizando App Router y JavaScript. Las carpetas y los archivos se irán creando a medida que se implementen los módulos.

```
offero/
├── public/
│   ├── images/
│   └── icons/
├── src/
│   ├── app/
│   │   ├── layout.js
│   │   ├── page.js
│   │   ├── globals.css
│   │   ├── login/
│   │   │   └── page.js
│   │   ├── registro/
│   │   │   └── page.js
│   │   ├── negocios/
│   │   │   ├── page.js
│   │   │   └── [businessId]/
│   │   │       └── page.js
│   │   ├── solicitudes/
│   │   │   ├── page.js
│   │   │   ├── nueva/
│   │   │   │   └── page.js
│   │   │   └── [requestId]/
│   │   │       └── page.js
│   │   ├── cotizaciones/
│   │   │   └── [requestId]/
│   │   │       └── page.js
│   │   ├── mensajes/
│   │   │   └── page.js
│   │   ├── perfil/
│   │   │   └── page.js
│   │   ├── administrador/
│   │   │   └── page.js
│   │   └── api/
│   ├── components/
│   │   ├── layout/
│   │   ├── ui/
│   │   ├── businesses/
│   │   ├── requests/
│   │   └── quotes/
│   ├── lib/
│   │   └── supabase/
│   ├── services/
│   ├── hooks/
│   └── utils/
├── .env.local
├── .gitignore
├── jsconfig.json
├── next.config.js
├── package.json
└── README.md
```

Importante: esta estructura es una guía de organización, no una lista de archivos que deban crearse todos desde el primer día. La versión concreta de Next.js puede generar archivos de configuración diferentes. Debemos conservar los archivos que cree el instalador y adaptar esta estructura al proyecto real.

Las credenciales y claves privadas deben mantenerse en variables de entorno y nunca subirse a GitHub. Las claves públicas de Supabase también deben utilizarse con las políticas de acceso correctamente configuradas.

8. Módulos

Un módulo solo se considera terminado cuando funciona correctamente en la versión publicada, sus operaciones principales han sido probadas y no depende de datos ficticios para simular su funcionamiento.

#	Módulo	Estado	Publicado	Responsable
0	Página mínima publicada	Pendiente	No	Diego

#	Módulo	Estado	Publicado	Responsable
1	Autenticación, registro y roles	Pendiente	No	Iker
2	Perfiles de negocios y categorías	Pendiente	No	Daniel
3	Publicación y gestión de solicitudes	Pendiente	No	Diego
4	Cotizaciones y comparación de propuestas	Pendiente	No	Iker
5	Mensajería y comunicación entre usuarios	Pendiente	No	Daniel
6	Reseñas y valoraciones	Pendiente	No	Daniel
7	Panel de administración y moderación	Pendiente	No	Diego

Estados posibles: Pendiente, En curso, Terminado.

Módulo 0 — Página mínima publicada

- Crear el proyecto con Next.js.
- Configurar React y Tailwind CSS.
- Preparar el repositorio de GitHub.
- Publicar una página básica en Vercel.
- Comprobar que la aplicación publicada carga correctamente.

Módulo 1 — Autenticación, registro y roles

- Registro e inicio de sesión con Supabase Auth.
- Perfil básico de usuario.
- Roles de cliente, proveedor y administrador.
- Protección de las operaciones según los permisos correspondientes.

Módulo 2 — Perfiles de negocios y categorías

- Crear y editar perfiles comerciales.
- Mostrar nombre, descripción, categoría, ubicación y datos de contacto.
- Consultar negocios y clasificarlos por categoría.

Módulo 3 — Publicación y gestión de solicitudes

- Crear solicitudes con descripción, cantidad, ubicación y fecha límite.
- Consultar solicitudes y sus detalles.
- Gestionar el estado de las solicitudes.
- Permitir que el comprador administre sus publicaciones.

Módulo 4 — Cotizaciones y comparación de propuestas

- Permitir que los proveedores envíen cotizaciones.
- Incluir precio, plazo de entrega, condiciones y vigencia.
- Mostrar las propuestas recibidas por solicitud.
- Facilitar la comparación y la selección de una propuesta.

Módulo 5 — Mensajería y comunicación entre usuarios

- Permitir la comunicación relacionada con una solicitud.
- Guardar los mensajes en PostgreSQL.
- Evaluar Supabase Realtime para actualizar los mensajes en tiempo real.
- Restringir la lectura y el envío de mensajes a usuarios autorizados.

Módulo 6 — Reseñas y valoraciones

- Permitir valoraciones después de una experiencia válida.

- Mostrar puntuaciones y comentarios de los negocios.
- Evitar reseñas duplicadas o no autorizadas mediante reglas de negocio.

Módulo 7 — Panel de administración y moderación

- Consultar y gestionar usuarios.
- Revisar publicaciones y reportes.
- Administrar categorías.
- Moderar contenido y aplicar las reglas de la plataforma.
- Restringir las funciones administrativas a usuarios autorizados.

9. Módulo actual

[En qué módulo están trabajando y en qué tarea van. Completar entre todos los integrantes.]

10. Problemas pendientes

- [Registrar aquí los errores o problemas que todavía no se han resuelto.]
- [Indicar los archivos afectados, el mensaje de error y los pasos para reproducirlo cuando sea posible.]
- [No marcar como resuelto un problema hasta comprobar que la solución funciona.]

11. Decisiones importantes

- 21/10/2026: El nombre del proyecto será Offero.
- 21/10/2026: La plataforma se enfocará en conectar compradores y proveedores mediante solicitudes y cotizaciones comparables.
- 21/10/2026: Se eligió Next.js con React y Tailwind CSS para el frontend.
- 21/10/2026: Se eligió JavaScript como lenguaje inicial para facilitar el aprendizaje y mantener la implementación alineada con el nivel del equipo.
- 21/10/2026: Se utilizará la lógica de servidor y API de Next.js, sin crear inicialmente un backend independiente.
- 21/10/2026: Se eligió Supabase con PostgreSQL como base de datos y proveedor de servicios de autenticación y almacenamiento.
- 21/10/2026: Se utilizarán Git y GitHub para el control de versiones y Vercel para publicar la aplicación.
- 21/10/2026: Se priorizará un MVP antes de desarrollar funciones avanzadas, como pagos integrados, inteligencia artificial o una aplicación móvil nativa.
- 21/10/2026: El mercado inicial y la diferenciación comercial siguen pendientes de validación. B2B es una posibilidad por investigar, no una decisión definitiva.
- 21/10/2026: La monetización se evaluará mediante suscripciones, comisiones o pago por oportunidades comerciales, sin asumir que alguno de estos modelos ya está validado.